

ASESII 24A02 Project-Based Quiz 1

Ridge, Lasso, and Regularization for Neural Features

Group XX: [names and student IDs]

2026-07-13

Contents

Authorship and contribution statement	2
Abstract	2
Reproducible setup	2
Shared helper functions	3
1 Problem 1. Prediction design and leakage control	5
1.1 1A. Target and predictors	5
1.2 1B. Split and train-only preprocessing	6
1.3 1C. Training-data exploration	6
2 Problem 2. OLS, ridge, and lasso	6
2.1 2A. OLS baseline	6
2.2 2B. Ridge	6
2.3 2C. Lasso	7
2.4 2D. Training-based comparison and locked core model	7
2.5 2E. Perturbation or stability check	7
3 Problem 3. Mathematical mechanisms	7
3.1 3A. Ridge and conditioning	7
3.2 3B. Lasso optimality and soft thresholding	7
3.3 3C. Connect algebra to evidence	7

4	Problem 4. Elastic net and a neural-feature bridge	8
4.1	4A. Research question and source map	8
4.2	4B. Elastic net on original predictors	8
4.3	4C. Fixed ReLU features	8
4.4	4D. Locked declaration and final holdout evaluation	8
4.5	4E. Deep-learning connection and limitations	9
5	Problem 5. Report quality and reproducibility	9
5.1	5A. Reproduction record	9
5.2	5B. Recommendation and limitations	9
5.3	5C. AI verification record	9
	Preflight and session information	9

Authorship and contribution statement

We confirm that every group member can explain the submitted analysis. AI use is reported in `GroupXX_AI_Log.csv`, and the group checked every AI-assisted item used in this report.

Student	ID	Main contributions	Verification performed
[Name]	[ID]	[Work]	[Check]

Abstract

Maximum 150 words: prediction problem, methods, main evidence, and recommendation.

Reproducible setup

```
required_packages <- c("tidyverse", "glmnet", "broom", "knitr")
missing_packages <- required_packages[
  !vapply(required_packages, requireNamespace, logical(1), quietly = TRUE)
]
if (length(missing_packages)) {
  stop("Install the required packages before rendering: ",
       paste(missing_packages, collapse = ", "))
}

library(tidyverse)
```

```

library(glmnet)
library(broom)
library(knitr)

group_number <- as.integer(params$group_number)
if (length(group_number) != 1L || is.na(group_number) || group_number < 1L) {
  stop("Set params$group_number to your assigned positive group number.")
}

seeds <- list(
  split = 240200L + group_number,
  cv = 240300L + group_number,
  features = 240400L + group_number
)

```

Shared helper functions

```

find_exam_data <- function(filename) {
  input <- knitr::current_input(dir = TRUE)
  input_dir <- if (length(input) && nzchar(input)) dirname(input) else getwd()
  candidates <- c(
    file.path(input_dir, "data", filename),
    file.path(input_dir, "..", "data", filename),
    file.path(getwd(), "data", filename)
  )
  existing <- candidates[file.exists(candidates)]
  if (!length(existing)) stop("Cannot locate ", filename, " by a relative path.")
  hits <- unique(normalizePath(existing, winslash = "/", mustWork = TRUE))
  if (length(hits) > 1L && length(unique(unname(tools::md5sum(hits)))) > 1L) {
    stop("Multiple nonidentical copies of ", filename, " were found.")
  }
  hits[[1L]]
}

split_rows <- function(n, train_prop = 0.80, seed) {
  stopifnot(n >= 10L, train_prop > 0, train_prop < 1)
  set.seed(seed)
  train <- sort(sample.int(n, floor(train_prop * n), replace = FALSE))
  list(train = train, test = setdiff(seq_len(n), train))
}

fit_scaler <- function(x, tol = 1e-12) {
  x <- as.matrix(x)
  center <- colMeans(x)
  centered <- sweep(x, 2L, center, "-")
}

```

```

scale <- sqrt(colMeans(centered^2))
keep <- is.finite(scale) & scale > tol
if (!any(keep)) stop("No nonconstant training predictor remains.")
list(
  center = center[keep],
  scale = scale[keep],
  keep = keep,
  dropped = colnames(x)[!keep]
)
}

apply_scaler <- function(x, prep) {
  x <- as.matrix(x)[, prep$keep, drop = FALSE]
  x <- sweep(x, 2L, prep$center, "-")
  sweep(x, 2L, prep$scale, "/")
}

make_foldid <- function(n, k = 5L, seed) {
  if (k < 3L || k > n) stop("Require 3 <= k <= number of training rows.")
  set.seed(seed)
  sample(rep(seq_len(k), length.out = n))
}

fit_cv_glmnet <- function(x, y, alpha, foldid) {
  glmnet::cv.glmnet(
    x = x,
    y = y,
    family = "gaussian",
    alpha = alpha,
    foldid = foldid,
    standardize = FALSE,
    intercept = TRUE,
    type.measure = "mse"
  )
}

score_regression <- function(y, prediction) {
  y <- as.numeric(y)
  prediction <- as.numeric(prediction)
  if (length(y) != length(prediction) || any(!is.finite(c(y, prediction)))) {
    stop("Metrics require equal-length finite vectors.")
  }
  c(RMSE = sqrt(mean((y - prediction)^2)),
    MAE = mean(abs(y - prediction)))
}

count_nonzero <- function(fit, s = "lambda.min", tol = 1e-8) {

```

```

b <- as.matrix(stats::coef(fit, s = s))
slopes <- b[rownames(b) != "(Intercept)", 1L]
sum(abs(slopes) > tol)
}

safe_condition_numbers <- function(x, lambda, tol = 1e-10) {
  eigenvalues <- eigen(
    crossprod(x) / nrow(x),
    symmetric = TRUE,
    only.values = TRUE
  )$values
  largest <- max(eigenvalues)
  smallest <- max(min(eigenvalues), 0)
  cutoff <- tol * max(1, largest)
  c(
    gram = if (smallest <= cutoff) Inf else largest / smallest,
    ridge = (largest + lambda) / (smallest + lambda)
  )
}

```

1 Problem 1. Prediction design and leakage control

1.1 1A. Target and predictors

```

config <- if (group_number %% 2L == 1L) {
  list(
    file = "fat.csv",
    response = "brozek",
    excluded = c("brozek", "siri", "density", "free")
  )
} else {
  list(
    file = "prostate.csv",
    response = "lpsa",
    excluded = "lpsa"
  )
}

data_raw <- read.csv(find_exam_data(config$file), check.names = FALSE)
predictors <- setdiff(names(data_raw), config$excluded)
analysis_data <- data_raw[, c(config$response, predictors), drop = FALSE]

if (anyNA(analysis_data)) stop("Declare a training-only missing-value plan.")
if (!all(vapply(analysis_data, is.numeric, logical(1)))) {

```

```
stop("The assigned response and predictors must be numeric.")
}
```

Define the prediction task and justify every exclusion.

1.2 1B. Split and train-only preprocessing

```
split <- split_rows(nrow(analysis_data), seed = seeds$split)
train_id <- split$train
test_id <- split$test

x_raw <- as.matrix(analysis_data[, predictors, drop = FALSE])
y_raw <- analysis_data[[config$response]]

x_prep <- fit_scaler(x_raw[train_id, , drop = FALSE])
x_train <- apply_scaler(x_raw[train_id, , drop = FALSE], x_prep)
x_test <- apply_scaler(x_raw[test_id, , drop = FALSE], x_prep)
y_train <- y_raw[train_id]
y_test <- y_raw[test_id]

foldid <- make_foldid(nrow(x_train), k = 5L, seed = seeds$cv)
```

Report seeds, dimensions, scaling, folds, and leakage safeguards.

1.3 1C. Training-data exploration

```
# Use analysis_data[train_id, ] only. Add focused plots and summaries.
```

Interpret multicollinearity, one anomaly, and the question regularization will address.

2 Problem 2. OLS, ridge, and lasso

2.1 2A. OLS baseline

```
# Fit OLS on x_train and y_train. Do not access y_test here.
```

2.2 2B. Ridge

```
# Use fit_cv_glmnet(..., alpha = 0, foldid = foldid).  
# Report lambda.min, lambda.1se, paths, coefficients, and CV evidence.
```

2.3 2C. Lasso

```
# Use fit_cv_glmnet(..., alpha = 1, foldid = foldid).  
# Report lambda.min, lambda.1se, paths, and nonzero predictors.
```

2.4 2D. Training-based comparison and locked core model

```
# Build a comparison from training/CV quantities only.
```

Core model nominated before holdout evaluation: [method and reason]

2.5 2E. Perturbation or stability check

Prediction before running the check: [prediction]

```
# Implement one allowed training-only stability check.
```

Observed result and explanation: [result]

3 Problem 3. Mathematical mechanisms

3.1 3A. Ridge and conditioning

Write the ridge derivation. Define dimensions and explain uniqueness.

```
# Use x_train and the fitted ridge lambda.min.  
# safe_condition_numbers(x_train, lambda = ...)
```

3.2 3B. Lasso optimality and soft thresholding

Derive the zero/nonzero optimality conditions and the orthonormal-design formula.

3.3 3C. Connect algebra to evidence

Connect one specific numerical result to Problem 3A or 3B.

4 Problem 4. Elastic net and a neural-feature bridge

4.1 4A. Research question and source map

Research direction: [weight decay / sparse weights / pruning / dropout]

Claim	Exact primary source and locator	What it establishes	What it does not establish
[Claim]	[Citation, section/page/equation]	[Support]	[Boundary]

4.2 4B. Elastic net on original predictors

```
alpha_grid <- c(0.10, 0.25, 0.50, 0.75, 0.90)

# Fit one cv.glmnet object per alpha with the same foldid.
# Select alpha and lambda using training CV only.
```

4.3 4C. Fixed ReLU features

```
relu <- function(z) pmax(z, 0)
M <- 200L

# TODO:
# 1. Generate A and bias once from seeds$features using the stated distributions.
# 2. Apply the same A and bias to x_train and x_test.
# 3. Fit a scaler to H_train only and apply it to H_test.
# 4. Fit ridge, lasso, and elastic net on H_train with the shared foldid.
```

State which weights are fixed, which are learned, and how active features are counted.

4.4 4D. Locked declaration and final holdout evaluation

Locked before viewing `y_test`: [preprocessing, candidate specifications, selected tuning values, feature seed, nominated deployment model, metrics]

```
# This must be the first analysis chunk that uses y_test for evaluation.
# Generate all fixed candidate predictions and one RMSE/MAE table here.
```

State whether the holdout evidence supports the predeclared choice. Do not retune.

4.5 4E. Deep-learning connection and limitations

Distinguish penalty, weight decay, output-weight sparsity, pruning, dropout, and a trained deep network. Cite the sources from 4A and discuss at least two limitations.

5 Problem 5. Report quality and reproducibility

5.1 5A. Reproduction record

1. Open a clean R session.
2. [Exact commands and folder layout.]
3. Render GroupXX_ProjectQuiz1.Rmd.

5.2 5B. Recommendation and limitations

Give the final recommendation, distinguish prediction from interpretation, and state limitations.

5.3 5C. AI verification record

AI-assisted item	How it was checked	Evidence	Modification or rejection
[Item]	[Method]	[Test/source/output]	[Decision]

Preflight and session information

```
# Add every object required to reproduce the final table and figures.
required_objects <- c(
  "train_id", "test_id", "x_train", "x_test", "y_train", "y_test", "foldid"
)
missing_objects <- setdiff(required_objects, ls())
if (length(missing_objects)) {
  stop(
    "Missing required objects: ",
    paste(missing_objects, collapse = ", ")
  )
}
stopifnot(length(intersect(train_id, test_id)) == 0L)
```

```
sessionInfo()
```

```

## R version 4.5.2 (2025-10-31)
## Platform: x86_64-pc-linux-gnu
## Running under: Ubuntu 26.04 LTS
##
## Matrix products: default
## BLAS:   /usr/lib/x86_64-linux-gnu/openblas-pthread/libblas.so.3
## LAPACK: /usr/lib/x86_64-linux-gnu/openblas-pthread/libopenblas-p0.3.32.so; LAPACK version
##
## locale:
##  [1] LC_CTYPE=en_US.UTF-8      LC_NUMERIC=C
##  [3] LC_TIME=en_US.UTF-8      LC_COLLATE=en_US.UTF-8
##  [5] LC_MONETARY=en_US.UTF-8  LC_MESSAGES=en_US.UTF-8
##  [7] LC_PAPER=en_US.UTF-8     LC_NAME=C
##  [9] LC_ADDRESS=C             LC_TELEPHONE=C
## [11] LC_MEASUREMENT=en_US.UTF-8 LC_IDENTIFICATION=C
##
## time zone: Asia/Ho_Chi_Minh
## tzcode source: system (glibc)
##
## attached base packages:
## [1] stats      graphics  grDevices  utils      datasets  methods   base
##
## other attached packages:
##  [1] knitr_1.51      broom_1.0.13    glmnet_5.0      Matrix_1.7-4
##  [5] lubridate_1.9.5 forcats_1.0.1   stringr_1.6.0   dplyr_1.2.1
##  [9] purrr_1.2.2     readr_2.2.0     tidyr_1.3.2     tibble_3.3.1
## [13] ggplot2_4.0.3   tidyverse_2.0.0
##
## loaded via a namespace (and not attached):
##  [1] generics_0.1.4    shape_1.4.6.1    stringi_1.8.7    lattice_0.22-9
##  [5] hms_1.1.4         digest_0.6.39    magrittr_2.0.5    timechange_0.4.0
##  [9] evaluate_1.0.5    grid_4.5.2       RColorBrewer_1.1-3 iterators_1.0.14
## [13] fastmap_1.2.0     foreach_1.5.2    backports_1.5.1   survival_3.8-6
## [17] scales_1.4.0      codetools_0.2-20 cli_3.6.6         rlang_1.2.0
## [21] splines_4.5.2     withr_3.0.2      yaml_2.3.12       otel_0.2.0
## [25] tools_4.5.2       tzdb_0.5.0       vctrs_0.7.3       R6_2.6.1
## [29] lifecycle_1.0.5   pkgconfig_2.0.3  pillar_1.11.1     gtable_0.3.6
## [33] glue_1.8.1        Rcpp_1.1.2       xfun_0.58         tidyselect_1.2.1
## [37] rstudioapi_0.18.0 farver_2.1.2     htmltools_0.5.9   rmarkdown_2.31
## [41] compiler_4.5.2    S7_0.2.2

```